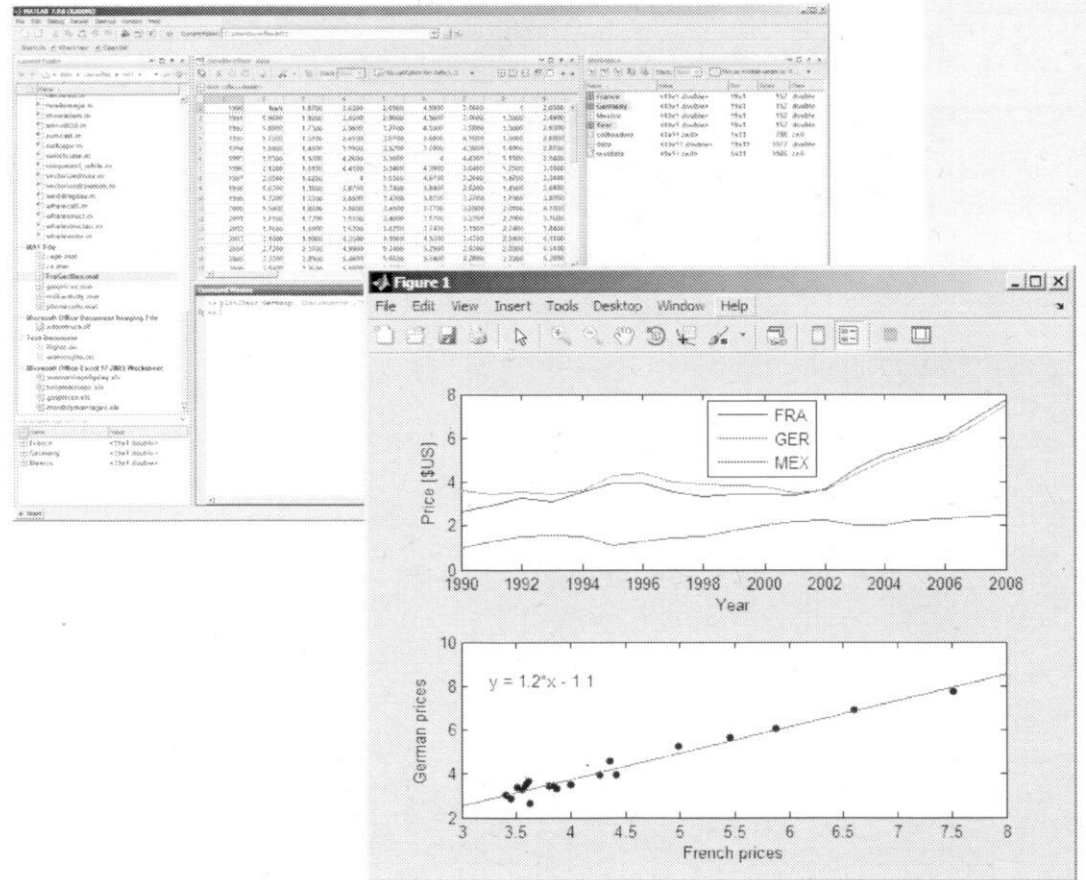


MATLAB foundation

0、 Working With the User Interface

Outline

- Reading data from file
- Saving and loading variables
- Plotting data
- Customizing plots
- Calculating statistics and best-fit line
- Exporting graphics for use in other applications



Example: Gas Price Data

Average Annual Gasoline (Petrol) Retail Prices in Selected Countries [\$US per gallon]

Source: US Dept. of Energy

<http://www.eia.doe.gov/emeu/aer/txt/ptb1108.html>

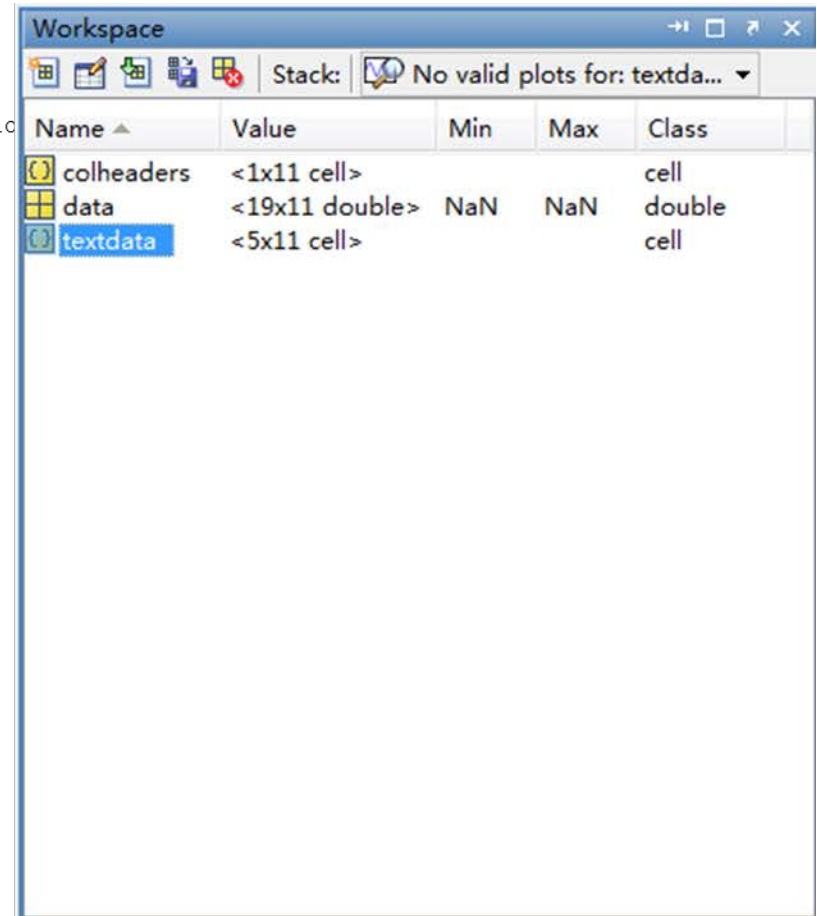
Year	Australia	Canada	France	Germany	Italy	Japan	Mexico	South Korea	UK	USA
1990		1.87	3.63	2.65	4.59	3.16	1.00	2.05	2.82	1.16
1991	1.96	1.92	3.45	2.90	4.50	3.46	1.30	2.49	3.01	1.14
1992	1.89	1.73	3.56	3.27	4.53	3.58	1.50	2.65	3.06	1.13
1993	1.73	1.57	3.41	3.07	3.68	4.16	1.56	2.88	2.84	1.11
1994	1.84	1.45	3.59	3.52	3.70	4.36	1.48	2.87	2.99	1.11
1995	1.95	1.53	4.26	3.96	4.00	4.43	1.11	2.94	3.21	1.15
1996	2.12	1.61	4.41	3.94	4.39	3.64	1.25	3.18	3.34	1.23
1997	2.05	1.62	4.00	3.53	4.07	3.26	1.47	3.34	3.83	1.23
1998	1.63	1.38	3.87	3.34	3.84	2.82	1.49	3.04	4.06	1.06
1999	1.72	1.52	3.85	3.42	3.87	3.27	1.79	3.80	4.29	1.17
2000	1.94	1.86	3.80	3.45	3.77	3.65	2.01	4.18	4.58	1.51
2001	1.71	1.72	3.51	3.40	3.57	3.27	2.20	3.76	4.13	1.46
2002	1.76	1.69	3.62	3.67	3.74	3.15	2.24	3.84	4.16	1.36
2003	2.19	1.99	4.35	4.59	4.53	3.47	2.04	4.11	4.70	1.59
2004	2.72	2.37	4.99	5.24	5.29	3.93	2.03	4.51	5.56	1.88
2005	3.23	2.89	5.46	5.66	5.74	4.28	2.22	5.28	5.97	2.30
2006	3.54	3.26	5.88	6.03	6.10	4.47	2.31	5.92	6.36	2.59
2007	3.85	3.59	6.60	6.88	6.73	4.49	2.40	6.21	7.13	2.80
2008	4.45	4.08	7.51	7.75	7.63	5.74	2.45	5.83	7.42	3.27

The Import Wizard

- Chose File→Import Data

Average Annual Gasoline (Petrol) Retail Prices in Selected Countries [\$US per gallon]
Source: US Dept. of Energy
<http://www.eia.doe.gov/emeu/aer/txt/ptb1108.html>

Year	Australia	Canada	France	Germany	Italy	Japan	Mexico	South Korea	UK
1990		1.87	3.63	2.65	4.59	3.16		1	2.05
1991	1.96	1.92	3.45	2.9	4.5	3.46		1.3	2.49
1992	1.89	1.73	3.56	3.27	4.53	3.58		1.5	2.65
1993	1.73	1.57	3.41	3.07	3.68	4.16		1.56	2.88
1994	1.84	1.45	3.59	3.52	3.7	4.36		1.48	2.87
1995	1.95	1.53	4.26	3.96	4	4.43		1.11	2.94
1996	2.12	1.61	4.41	3.94	4.39	3.64		1.25	3.18
1997	2.05	1.62	4	3.53	4.07	3.26		1.47	3.34
1998	1.63	1.38	3.87	3.34	3.84	2.82		1.49	3.04
1999	1.72	1.52	3.85	3.42	3.87	3.27		1.79	3.8
2000	1.94	1.86	3.8	3.45	3.77	3.65		2.01	4.18
2001	1.71	1.72	3.51	3.4	3.57	3.27		2.2	3.76
2002	1.76	1.69	3.62	3.67	3.74	3.15		2.24	3.84
2003	2.19	1.99	4.35	4.59	4.53	3.47		2.04	4.11
2004	2.72	2.37	4.99	5.24	5.29	3.93		2.03	4.51
2005	3.23	2.89	5.46	5.66	5.74	4.28		2.22	5.28
2006	3.54	3.26	5.88	6.03	6.1	4.47		2.31	5.92
2007	3.85	3.59	6.6	6.88	6.73	4.49		2.4	6.21
2008	4.45	4.08	7.51	7.75	7.63	5.74		2.45	5.83



The image shows a screenshot of the MATLAB Workspace window. It displays three variables imported from a file: 'colheaders' (a 1x11 cell array), 'data' (a 19x11 double array), and 'textdata' (a 5x11 cell array). The 'Stack' pane shows 'No valid plots for: textda...'. The 'Class' column indicates the data types: 'cell' for 'colheaders' and 'textdata', and 'double' for 'data'.

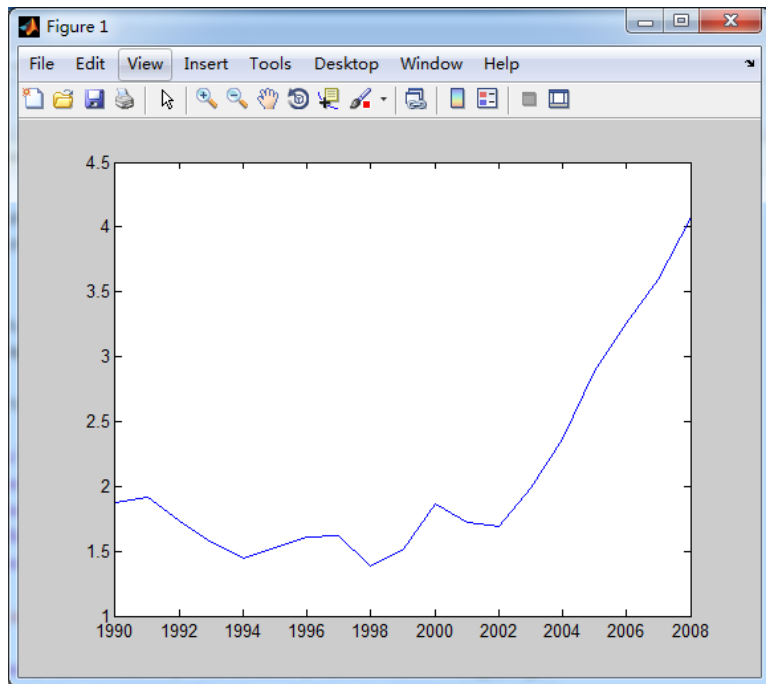
Name	Value	Min	Max	Class
colheaders	<1x11 cell>			cell
data	<19x11 double>	NaN	NaN	double
textdata	<5x11 cell>			cell

workspace

- The Variable Editor
- New Variables
- Saving and Loading

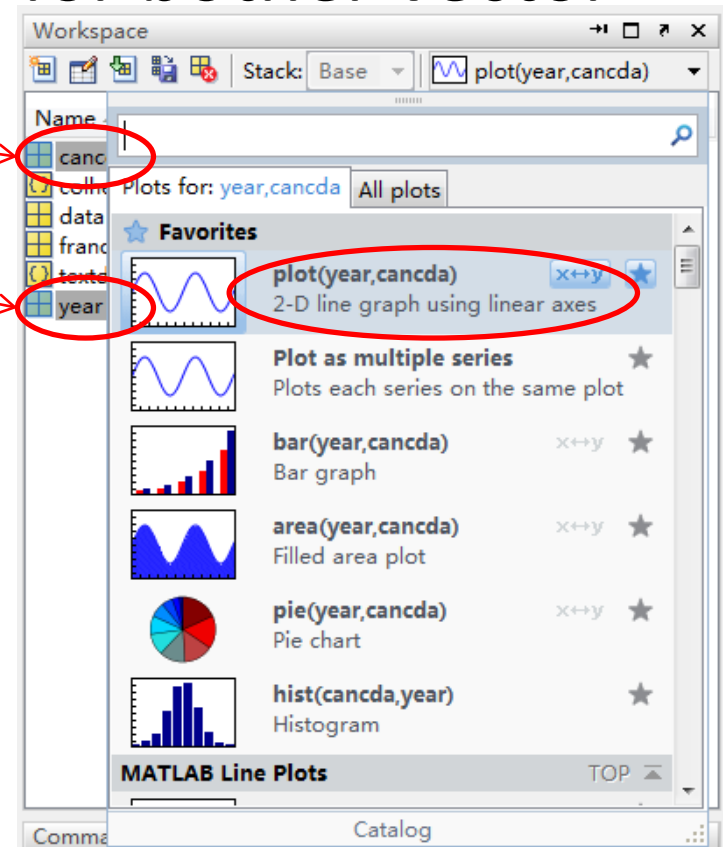
Plot and Data

- creating appropriate visualizations of data
- offer many plotting routine for both vector and matrix data

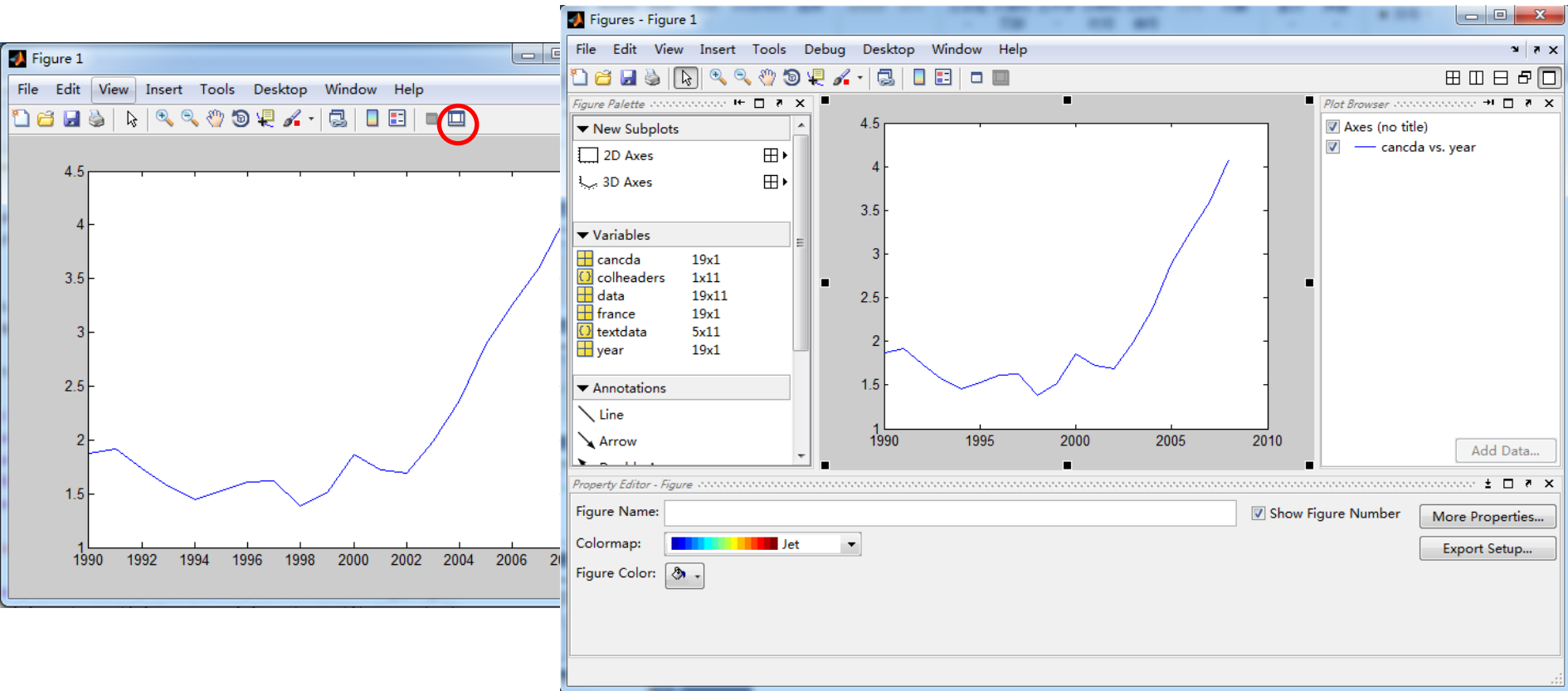


Ctrl+选
择canada

第一次选择

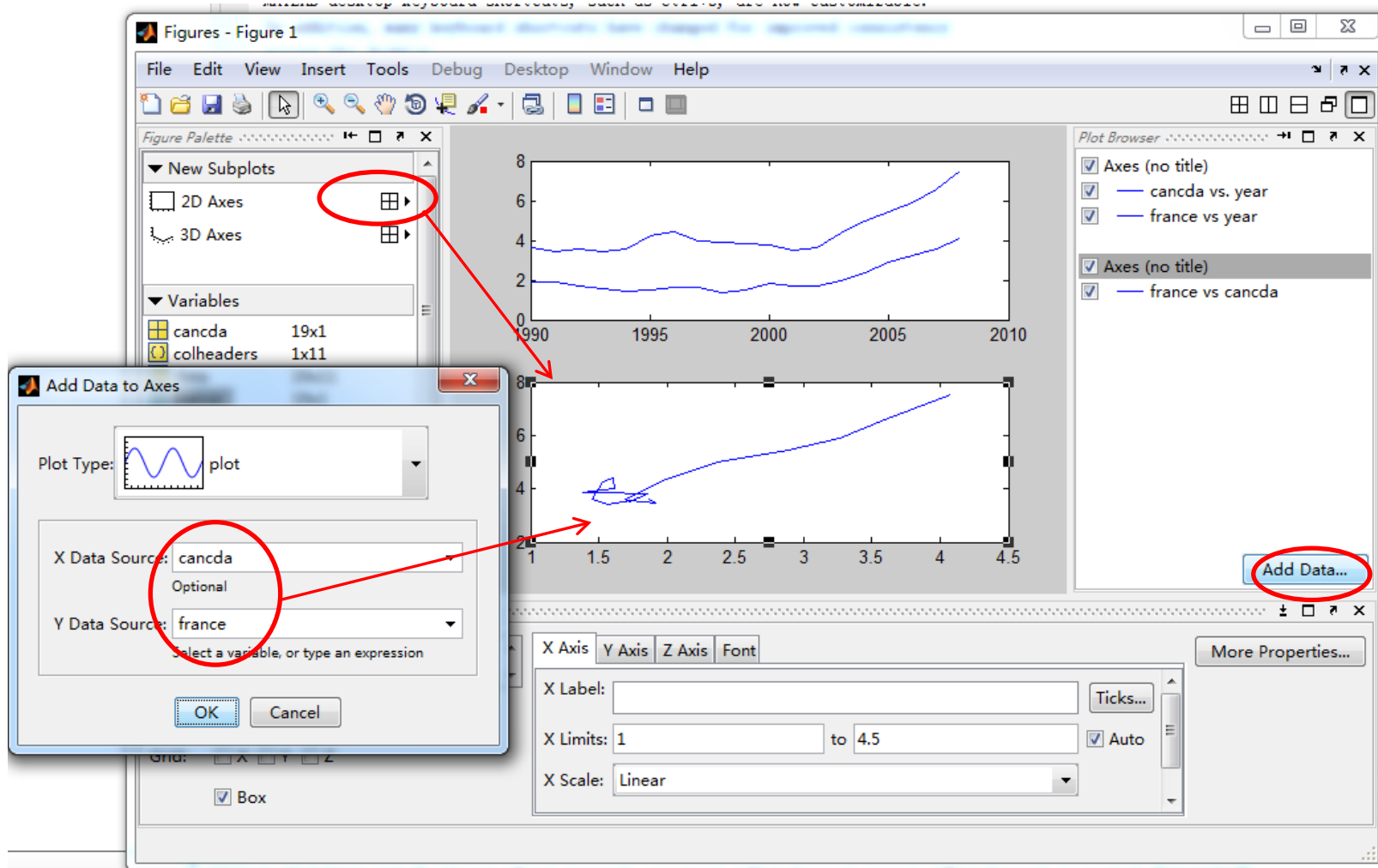


Plot Tools



- Add axes to create subplots within the figure window
- Drag and drop variable from the base workspace into any axes
- Add a variety of annotations of a plot

Multiple Plots

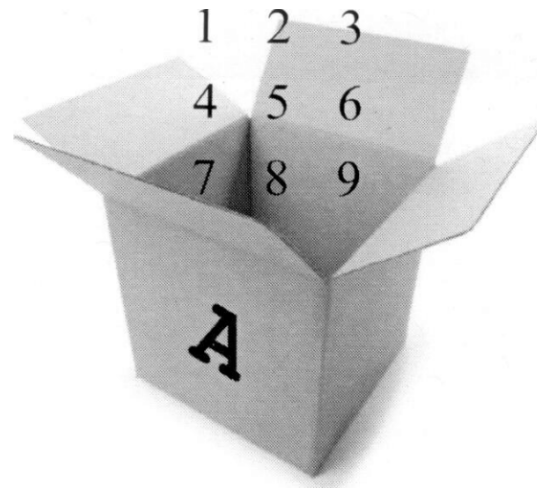


- *clear* clears base workspace variable from memory
- *clc* clears the command windows
- *close* closes the current figure

一、Variables and Expressions

Outline

- Entering commands
- Creating variables
- Getting help
- Accessing and modifying values in variables
- Creating character variables



MATLAB[®] Commands

- MATLAB is **case sensitive**
- MATLAB commands are entered in the Command Window at the MATLAB prompt(>>). Commands are executed once the **Enter** key is pressed.
- A semicolon(;) at the end of a command suppresses any output
- A number of keyboard shortcuts are available to assist in entering commands:
 - ↑, ↓ Move to the beginning , end of an expression
 - Esc Deletes the expression at the prompt
 - Tab Displays commands that complete a partial expression
- Useful commands for managing the Command Window are:
 - clc clears the Command Window
 - home Moves the prompt to the top of the Command Window
(without clearing commands)

Saving and Loading MAT-Files

- Variables can be saved to the native MATLAB format (MAT-files) directly from the Command Window using the save command:

```
>> save gasprices country gpdata
```

```
>> save filename variable1 variable2 ...
```

saves the two variables country and gpdata to the file `gasprices.mat` in the current directory.

- Saved variables can be loaded back to the workspace using the load command. You can specify particular variables to load, or load all variables in the MAT-file:

```
>> load gasprices gpdata
```

loads just the variable gpdata, whereas

```
>> load gasprices
```

loads everything in gasprices.mat.

Assignment

- Variables can be created in MATLAB using the assignment equality

```
>> x = 1 - 1i, y=pi;
```

- MATLAB variable names
 - must start with a letter
 - Contain (a-z and A-Z), number (0-9), and the underscore (_).
 - They can be anywhere from 1 to 63 characters long.

Assignment(cont')

- There is no requirement in MATLAB to declare the name, type, or size of a variable before making an assignment statement.
 - numerical data are created as double arrays;
 - each data value in memory to double (64 bit, 8 byte) precision.

```
>> z = 3
```

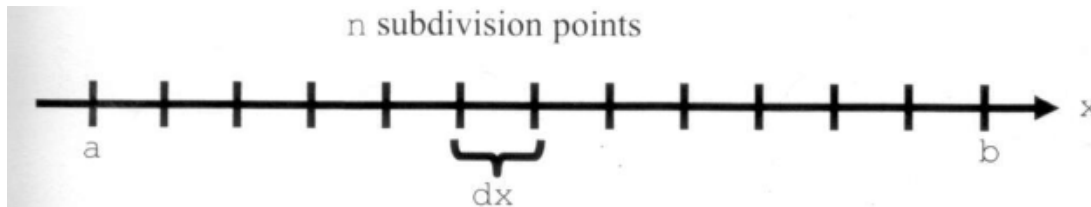
creates a double-precision value.

```
>> z = 5
```

overwrites z with a complex double-precision value.

Creating Vectors

- Vectors, or 1-dimensional arrays, are the basis for computing and displaying sequences of data in MATLAB.
- There are two basic methods for creating equally spaced vectors.



- If you know the size of the subdivisions dx between values, use

```
>> x = a:dx:b;
```

If dx is not specified, it is assumed to be 1.

- If you know the number of values (subdivision points) n , use the `linspace` function:

```
>> x = linspace(a,b,n);
```

If n is not specified, it is assumed to be 100.

Creating Vectors(cont')

- There are methods for creating nonequally spaced vectors.
 - Vectors of arbitrary data ---a row vector
`>> x=[1, 2, 3, 4, pi, 7]`
 - Vectors of data---column vector
`>>x=[cos(0);0;-42;7]`
 - Transpose a row vector
`>> y=x'`

Creating Matrices

```
>> A = [1, 2, 3; 4, 5, 6; 7, 8, 9];
```

or

```
>> A = [1 2 3; 4 5 6; 7 8 9];
```

or

```
>> A = [ 1 2 3
```

```
4 5 6
```

```
7 8 9]
```

Matrix Creation Functions

- MATLAB has a large number of matrix creation functions that output variables with certain characteristics and avoid tedious data entry.
- The basic syntax for matrix creation functions is:

```
>> A = function_name(m, n);
```

or

```
>> A = function_name(n)
```

- `zeros` creates a matrix with all values initialized to zero.
- `Ones` creates a matrix with all values initialized to one.
- `rand` creates uniformly distributed numbers between 0 and 1
- `randi` creates uniformly distributed integers
- `randn` creates normally distributed random numbers from the standard normal distribution (mean 0 and standard deviation 1).

Help and Documentation

MATLAB help and documentation can show you

- The various ways to call a function
- The algorithm implemented by a function
- Examples of how to use a function
- Links to related functions
- Tutorials and background information

To see the help on a particular function, such as `rand`, type

```
>> help rand
```

For complete documentation for the function, type

```
>> doc rand
```

help, doc, docsearch

Row, Column Indexing

- Row and column index begins with 1
- Array stores in column priority
- Data in MATLAB variables is accessed using one of several methods of indexing.
 - row, column indexing
 - >> A(m, n)
 - points MATLAB to the mth row and nth column of the variable A.
 - >> x = A(m, n);
 - MATLAB keyword end : the end of either a row or a column
 - >> x=A(3, end)
 - A single index
 - >> x=A(4)

Multiple Row, Column Indices

- You can reference multiple elements of an array using vectors as indices

```
>> x = A([1,2,5],[2,3]);
```

creates a 3-by-2 matrix of six elements of A: A(1,2), A(1,3), A(2,2), A(2,3), A(5,2), and A(5,3).

- You can indicate a range of rows or columns in a variable using the colon (:) notation

```
>> Ger90s = gpdata(1:10, 5);
```

extracts a portion of the fifth column of gasprice, and

```
>> Year = gpdata(1:end, 1);
```

extracts the entire first column.

Indexed Assignment

- *subscripted assignment*

>> A(m,n) = x;

it assigns the value x to the (m,n) element of A.

- Referencing and assignment can be combined in a single statement:

>> A(3,7) = 4 * A(3, 7);

replaces the (3,7) element of A with 4 times its current value.

Concatenation

- square brackets ([]) to delimit the start and end of an array.
- The square brackets are actually the *concatenation operator* in MATLAB. Recall that even scalar variables in MATLAB are 1-by-1 arrays. Hence, when you enter

```
>> A = [1, 2, 3; 4, 5, 6; 7, 8, 9];
```

- *concatenation operator*

```
>> A = [1, 2, 3; 4, 5, 6; 7, 8, 9];
```

```
>> B = [A; 10, 11, 12];
```

```
>> C = [A, [12, 34, 45]'] or
```

```
>> C = [A, [12; 34; 45]]
```

```
>> not correct C = [A, 10; 11; 12]
```

Characters and Strings

```
>> y = x          % variable
```

```
>> y = 'x'        % character
```

Try

```
>> s1 = 'Gas Prices'
```

```
>> s2 = lower(s1)
```

```
>> s3 = '.'
```

```
>> s4 = 'xls'
```

```
>> s5 = [s2,s3,s4]
```

Cell Arrays

- MATLAB arrays must always have the same number of columns in each row(and vice versa)
- A single string is treated as a 1-by-n array of characters.
`>> x = ['hello'; 'ciao'; 'guten tag']----- error`
 - Vertical concatenation of strings fails unless they are all the same length.
- To concatenate elements into a [cell array](#), use curly braces (`{ }`) instead of square brackets:
`>> x = {'hello'; 'ciao'; 'guten tag'}----correct, 3*1,each container has 1*n.`
`>> y = {pi, []; 'foo', 42; ones(2), eye(3)}`
`>> z = {x, y}`
`>> z = {x; y}`
`>> z = [x, y] % ok`
`>> z = [x; y] % no! dimension mismatch`

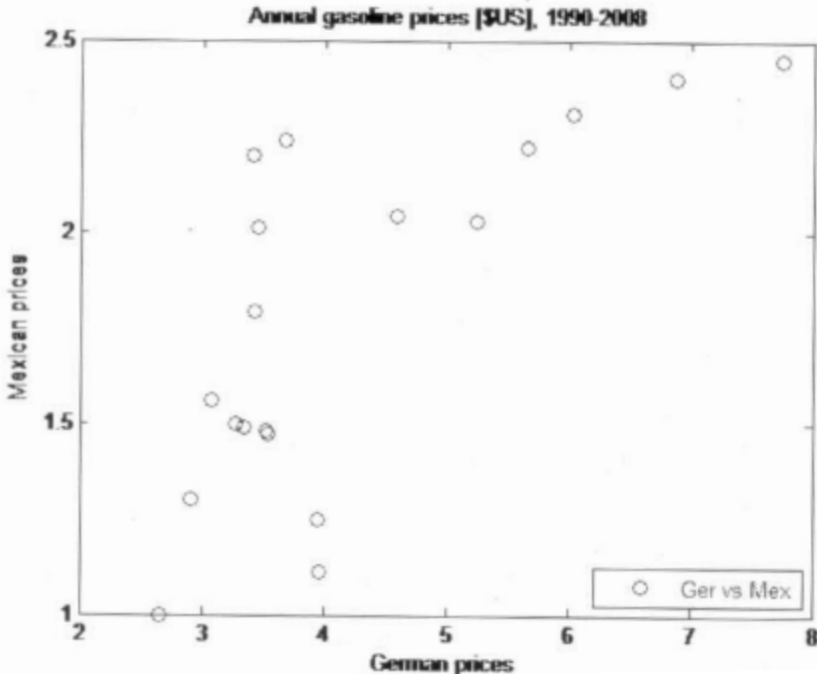
Cell Indexing

- Indexing into cell arrays is very similar to indexing into any other type of array.
- MATLAB provides two ways to index into cell arrays:
 - use parentheses to reference the container
`>> a = x(1) % a is a 1-by-1 cell array (the container),`
 - use curly braces to reference the contents:
`>> b = x{1} % b is a 1-by-5 character array (the contents).`

二、 Analysis and Visualization with Vectors

Outline

- Calculations with vectors
- Plotting vectors
- Basic plot options
- Annotating plots



$$\begin{bmatrix} 2.65 \\ 2.90 \\ 3.27 \\ 3.07 \\ 3.52 \\ 3.96 \\ 3.94 \\ 3.53 \\ 3.34 \\ 3.42 \\ 3.45 \\ 3.40 \\ 3.67 \\ 4.59 \\ 5.24 \\ 5.66 \\ 6.03 \\ 6.88 \\ 7.75 \end{bmatrix} + \begin{bmatrix} 1.00 \\ 1.30 \\ 1.50 \\ 1.56 \\ 1.48 \\ 1.11 \\ 1.25 \\ 1.47 \\ 1.49 \\ 1.79 \\ 2.01 \\ 2.20 \\ 2.24 \\ 2.04 \\ 2.03 \\ 2.22 \\ 2.31 \\ 2.40 \\ 2.45 \end{bmatrix} = \begin{bmatrix} 3.65 \\ 4.20 \\ 4.77 \\ 4.63 \\ 5.00 \\ 5.07 \\ 5.19 \\ 5.00 \\ 4.83 \\ 5.21 \\ 5.46 \\ 5.60 \\ 5.91 \\ 6.63 \\ 7.27 \\ 7.88 \\ 8.34 \\ 9.28 \\ 10.20 \end{bmatrix}$$

Statistical Functions

- Basic statistical functions in MATLAB can be applied to a vector of data to produce a single output:
 >> GerMean = mean(Germany)
 >> MaxStart = max(StartPrices)
- Some of these functions can return extra optional arguments.
 >> [MaxMex, MMIndex] = max(Mexico);
- Note the use of square brackets (concatenation) construction for multiple function outputs

Array Operations

- MATLAB is designed to work with matrices and vectors as mathematical objects.
- Matrix addition and subtraction is defined as an element-by-element operation --- this is, if

```
>> C = A + B
```

- Then $C(k)$ is $A(k)+B(k)$. Similarly, scalar multiplication or division of an array is defined mathematically:

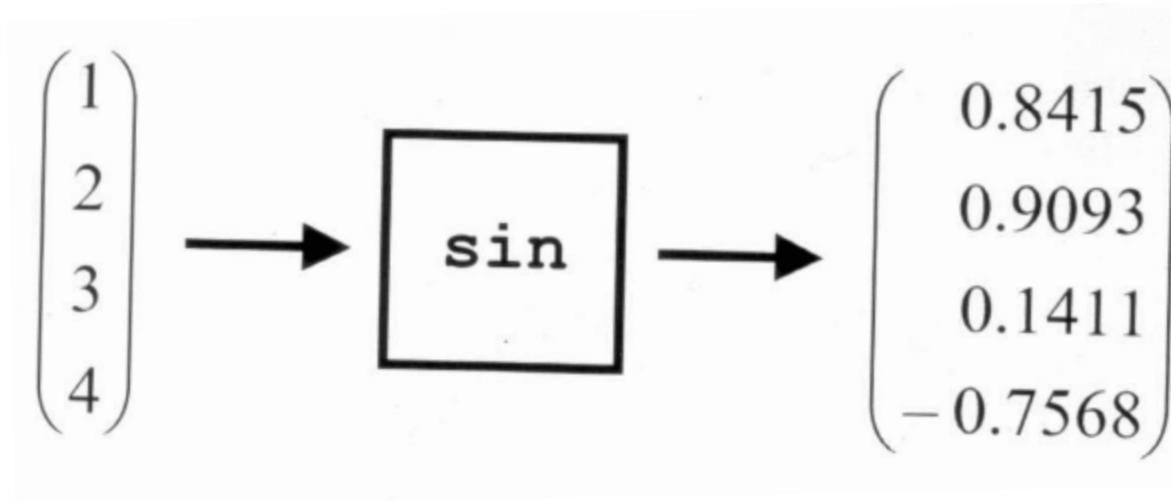
```
>> B = 2*A
```

Multiplies each element of A by 2.

Mathematical Operations

Performed on all elements of a vector

$$\sin([a \ b \ c]) = [\sin(a) \ \sin(b) \ \sin(c)]$$



sin

sin d

sinh

a sin

exp

log

log 2

log10

sqrt

abs

angle

floor

ceil

round

mod

Plotting Vectors

- The vectors of the same length can be plotted against each other with the plot function:

```
>> plot(x,y)
```

- This plots each element of x on the horizontal axis against the corresponding element of y on the vertical axis.

Neither vector is stored before plotting.

- If only a single input vector is passed to plot, the vector is plotted on the vertical axis, using the array index as the horizontal axis.

Plot Options

- The plot function accepts an optional third argument that allows you to specify the basic properties of the plot's appearance: line style, line color, and marker style.

- The codes can be combined in any order, so

```
>> plot(Year, Mexico, 'k^:')
```

and

```
>> plot(Year, Mexico,':^k')
```

Both plot Mexican gas prices with a black dotted line and triangular markers.

Color		Marker		Line style	
b	blue	.	point	-	solid
g	green	o	circle	--	dashed
r	red	x	(diagonal) cross	:	dotted
c	cyan	+	(vertical) cross	-.	dash-dot
m	magenta	*	star		
y	yellow	s	square		
k	black	d	diamond		
w	white	v	triangle (down)		
		^	triangle (up)		
		<	triangle (left)		
		>	triangle (right)		
		p	pentagram		
		h	hexagram		

Note If a marker style is given, but no line style is given, no line will be plotted.

Annotating Plots

- Textual information can be added to plots using separate annotation functions: title, xlabel, ylabel, and legend. All functions take a string input:

```
>> title('Annual gasoline prices')
```

```
>> xlabel('German prices')
```

```
>> ylabel('Mexican prices')
```

```
>> legend('Ger vs Mex')
```

Additional Vector Plot Types

- MATLAB has many specialized plotting commands to allow you to visualize and present your data with exactly the right emphasis on the data features you find important. Some of the additional plot types for vector data include

<code>bar</code>	Bar graph (vertical and horizontal)
<code>stem</code>	Discrete sequence (signal) plot
<code>stairs</code>	Stairstep graph
<code>area</code>	Filled area plot
<code>pie</code>	Pie chart
<code>hist</code>	Histogram